从Prompt到Context：为什么Think Tool是形式化的必然？

凤聆 大淘宝技术 2025年10月9日 01:57 浙江 15人



本文以编译原理中的语言形式化理论（如乔姆斯基谱系）为基础，指出AI领域从Prompt Engineering（低形式化，依赖语言微调，脆弱且难扩展）向Context Engineering（中等形式化，借RAG、工具集成等结构化方式提升可靠性）的演进，本质是为获取系统可追踪性与可靠性而牺牲LLM部分无限制表达能力；同时分析Anthropic的Think Tool，其通过形式化工具定义将模型内部推理转化为显式可验证动作，超越非形式化的CoT，最后提出未来需为Agent System构建全面形式化理论，以实现AI智能体行为的精确规约与验证，满足高风险领域需求，体现了语言形式化在AI思想层面的新发展。

01 编译原理的语言形式化

1.1 精确性定义的必要性

语言形式化是以数学的严谨性来定义一种语言的语法（结构）和语义（意义），从而消除歧义的过程。大家需要注意这个不是纯粹的学术探讨，而是构建可靠系统的实践需求。**最终目标是实现“更高的可靠性、可读性和可维护性，并降低开发成本”**。这之所以可能，是因为有了所谓的更精确的语言定义。

一个形式化的语言定义为程序员提供了精确的规范，确保了编译器之间的兼容性，并使其属性得以进行严格分析，**如果缺乏形式化，语言的构造将变得错综复杂，对属性的证明也会复杂而微妙**。

整个过程涉及通过属性语法（attribute grammar）等形式化方法来指定语法和语义，这让编译器模块的自动生成理论上成为可能，进一步，编译器正确性的黄金标准是“语义保持”（semantic preservation），即确保编译后程序（目标语言）的行为与原始程序（源语言）的语义兼容，这通常通过执行迹线（execution traces）来验证。**这种可验证迹线的概念，我们也会介绍在Anthropic近期提出来的，我也在文章写过的Think Tool上的实践模式，这个**

从根源上也来自于这样的连接概念，对于我们发现一些新的方法论和有效的工具，非常有帮助。

1.2 乔姆斯基谱系：表达能力与约束的规范化标尺

20世纪50年代被乔姆斯基清晰进行了定义，定义了不同的语言的形式化程度，这些形式化程度也从理论上说明计算机对这种的表达和可追踪的程度。乔姆斯基谱系为语言的形式化程度提供了一个天然的、分级的标尺，这个标尺从0型文法到3型文法，产生式规则变得越来越受约束，这降低了语言的表达能力，但极大地提高了可预测性以及识别（或者说解析）的效率。

这个谱系是一个形式文法类别的包含谱系，从0型（无限制）到3型（正则），每一层级都是其上一层级的真子集，看这个图表：



文法类型	语言类别	识别自动机	产生式规则形式	关键特性	在计算中的主要应用
0型	递归可枚举语言	图灵机	α→β (其中 α 包含至少一个非终结符)	不可判定	计算理论、算法复杂性
1型	上下文有关语言	线性有界自动机	αAγ→αβγ (其中 γ 非空)	可判定，但复杂度高	自然语言处理（部分）、生物列建模
2型	上下文无关语言	下推自动机	A→α (其中 A 是单个非终结符)	可高效解析（多项式时间）	编程语言的语法分析、解析器成
3型	正则语言	有限状态自动机	A→aB 或 A→a	线性时间识别、闭包性质良好	词法分析、文本搜索（正则表式）、协议定义

了解谱系结构，可以了解到语言表达能力与计算可追踪性之间的矛盾，当规则的约束增强时（从0型走向3型），语言的表达范围收窄，但其分析、解析和验证的难度显著降低。反之，当约束放宽时，语言变得无所不能，但其行为也变得不可预测。基于乔姆斯基谱系，语言的形式化程度可被定义为，一种语言的产生式规则受约束的程度，该程度决定了其在可预测性、可验证性和计算可追踪性谱系中的位置。更高的形式化程度（如3型文法）意味着更严格的规则、更弱的表达能力，但具备高度的可预测性和高效的处理能力。更低的形式化程度（如0型文法）则意味着极大的表达能力，但以牺牲可预测性和可验证性为代价。

这个权衡，到今年2025年，我发现其实就是AI工程师所面临的困境，从Prompt Engineering到Context Engineering的一个转变。本质上是一个为了获得结构化系统的可追踪性和可靠性，而有意识地牺牲原始LLM部分无限制表达能力的决策，所以编译器设计的历史教训正在AI领域重演，很有意思，我们后面会讲一讲为什么。

02 Prompt和Context Engineering在形式化上的分析

首先在这里说到的语言，并不是严格按照上面的谱系语言分级，也不是编程语言，这里指的是整个人类的自然语言，而这里的编译器指的是LLM，相当于是LLM来编译人类的语言产生结果，这之间自然是存在类似的形式化分级。

2.1 Prompt Engineering：非形式化规约的艺术（低形式化）

Prompt其实位于形式化谱系的低端，类似于0型或1型文法，交互依赖于语言学上的微调和寻找魔法词汇，或者遵从很结构化的玩法，产生的技巧就是角色定义、few-shot examples、Chain-of-Thought以及各种格式约束。

这种玩法的核心弱点在于脆弱性和缺乏可扩展性，措辞的微小变化就可能导致输出质量的巨大差异，这样其实很不适用于生产级系统，所以并不是让那个系统去流利地讲LLM语言这个理念，这个感觉下来，Prompt Engineering，更像是一门艺术（笑），而不是一门科学手艺。

2.2 Context Engineering：结构化系统交互的兴起（中等形式化）

Context Engineering代表了向更高形式化程度的重大转变，用结构化的、机器可读的上下文取代了模糊的自然语言指令。它不再将LLM视为一个需要被说服的对话者，而是将其作为一个更庞大的信息处理流水线中的一个组件。

这块技术是架构层面，采用的有RAG、工具集成、结构化模板和记忆管理，涉及到从数据库、API和对话历史中筛选、聚合和格式化信息。目标是在正确的时间，以正确的格式，提供正确的信息和工具，这种结构化的方法减少了幻觉，并实现了可靠的多轮交互。所以这种Context涉及系统思维和构建智能系统架构，**核心是提供“使LLM plausibly 解决任务所需的所有上下文”**，这种上下文是动态的，由一个在LLM调用之前运行的系统来生成出来的。

2.3 两者比较分析

从Prompt Engineering到Context Engineering的演变，其实是将LLM从纯粹的黑箱，转变为将其集成为一个灰箱组件的过程，这个转变的模式，其实跟从汇编语言的位操作演进到高级结构化编程的历程异曲同工。Prompt告诉模型如何思考，而Context则为模型提供完成工作所需的训练和工具。



这种转变也重新定义了对工程人员的新的挑战，因为模式已经发生了变化，现在LLM的上下文窗口有点像一个RAM，Context Engineering把有效管理这块有限的RAM的玩法，用摘要、检索和记忆分槽（短期、长期）来加强，**其实从计算机科学上，也就是当年在解决传统操作系统中的内存管理问题**，比如决定哪些数据应保留在快速访问的内存中（把数据加载到Context窗口），哪些应换出到慢速存储（如Vector DB），以及在需要时如何高效地将其取回。

Prompt的脆弱性，部分源于其糟糕的内存管理，模型在长对话中无法记住关键信息，Context Engineering通过建立一个显式的内存架构来解决这个问题。这意味着高级AI工程所需的技能正在与经典系统架构的技能趋同，**重点是数据流、状态管理和资源优化，而不仅仅是语言学。**

维度	Prompt Engineering	Context Engineering
核心方法论	通过精心设计的文本指令来引导模型。	构建动态系统，为模型提供信息、工具和记忆。
形式化程度	0/1型文法，依赖LLM隐式的、不透明的语法。	2/3型文法，使用结构化数据、API模式和形式化的工具定义。
主要目标	在单次交互中，通过说服模型来优化输出。	在复杂任务中，通过系统性给模型来确保可靠性和一致性。
关键技术	少样本示例、思维链、角色扮演、格式约束。	RAG、工具使用、记忆管理、动态上下文组装。

可扩展性与可靠性	低+脆弱，对微小变化敏感，难以维护和扩展。	高+稳健，通过结构化输入降低不确定性，支持复杂、多轮次的应用。
所需技能	语言创造力、对特定模型行为的直觉理解。	系统设计、信息架构、数据策略、API集成。
控制点	用户与模型之间的单一文本接口。	模型调用之前的整个信息准备流水线。
类比	与一位才华横溢但喜怒无常的专家对话。	为一位专家配备一个藏书丰富的图书馆和一个全能的工具箱。

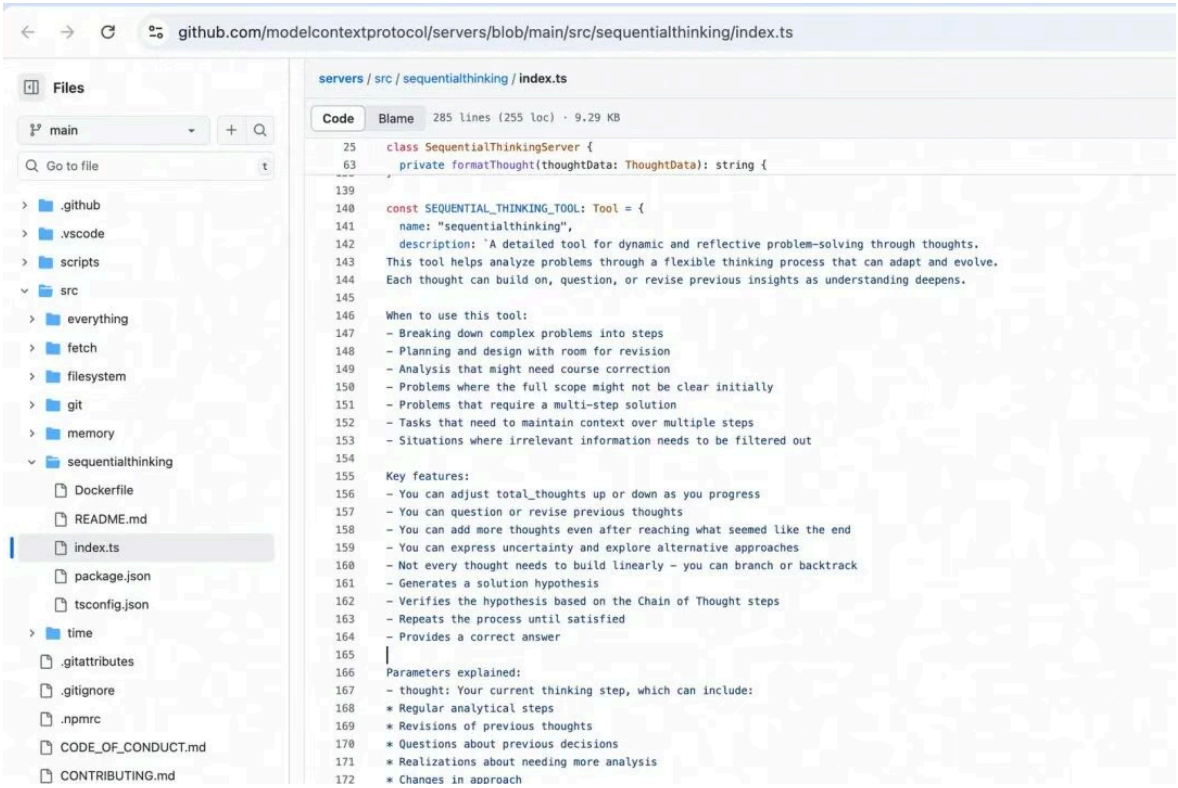
03 Anthropic的think tool分析

3.1 显卡推理的架构



Anthropic的think工具，简单来说，指的就是模型在意识到自己上下文不足的时候，可以调用一个思考工具，让自己自言自语一下，补充一些逻辑支撑。这块的设计提升的效果非常的明显，在几个案例上都达到了明显的提效，甚至比R1的预先的思考更有效，这种玩法就是把思考这个行为从一种隐式的、不可观测的过程，转变为系统执行迹线中一个显式的、结构化的、可验证的动作。

这个工具通过一个形式化的工具定义来实现，包含名称、描述和输入模式（input schema），比如mcp协议中标准的的think-tool定义：



它创建了一个专用于结构化思考的空间，也有很多人认为它并非传统意义上的工具（因为它不调用外部API），而更像是Claude API的一个特性或模式，这一细微差别非常重要，**它形式化的是一个内部认知过程，这比仅仅形式化对外部数据的访问要深刻得多。**

3.2 可验证性与策略遵循

think tool的结构化输出，功能类似于编译器中的中间表示（Intermediate Representation, IR）或程序的执行迹线，它使模型的推理过程变得可审计，从而可以根据复杂的规则和策略进行验证。

该工具在策略密集型环境（policy-heavy environments）中最为有效，在这些环境中，可以提示模型去列出适用于当前请求的具体规则、检查是否已收集所有必需信息”以及“验证计划中的行动是否符合所有策略，这直接呼应了编译器设计中“语义保持”的目标，即必须证明目标代码的行为（迹线）与源代码的语义相符，think tool产生的日志，正是模型进行语义推理的迹线。

性能提升在复杂领域（如航空公司订票系统）尤为显著（相对提升达54%），具体可以看我之前那篇文章，这里不赘述太多，这正是因为这些领域拥有复杂且相互依赖的规则，而显式的验证过程能带来巨大好处，这种机制增强了系统的透明度、可调试性和可审计性，这些都是高风险应用的关键要求。

3.3 对CoT的范式超越

think tool也代表了对非形式化的CoT的演进，CoT虽然能够引出推理过程，但它将推理与最终答案混杂在非结构化的文本中。think tool则将推理模块化、形式化，使其成为一个独立的、可被检视的步骤，所以CoT是一种提示技巧，一种语言引导方式，think tool则是一个形式化的API特性。

这个tool用于在响应生成期间暂停，以处理来自工具调用的新信息，而CoT通常是初始提示的一部分，用以构建整个输出的结构，这让think tool更适用于动态的、多步骤的任务，这种形式化实现了关注点分离，思考步骤用于内部思考，最终答案用于外部沟通，这种模块化是所有结构良好系统的标志。

这种机制为模型提供了一种元认知脚手架（metacognitive scaffolding），**元认知指的就是，对思考的思考，涉及规划、监控和评估自身的思维过程**。think tool的description明确要求模型执行这些元认知动作，将复杂问题分解、评估多种方法、验证推理是否存在逻辑错误，这些方法不仅仅是要求模型去推理，而是提供一个外部的、形式化的结构（工具调用及其日志），迫使模型遵循一个特定的元认知工作流。

还有一点，**这种形式化的、可检视的推理迹线是实现稳健的自我纠错的前提**，Agent面临的一个关键挑战是如何从错误中恢复。在一个长CoT中，一个错误就可能导致整个任务失败，think tool允许模型在调用工具后仔细分析工具输出。通过将推理和工具调用结果外部化到一个结构化日志中，模型（或控制它的系统）可以检查当前状态和导致该状态的推理过程。如果工具调用返回错误或非预期数据，模型可以再用一个think步骤来分析失败原因并规划新的行动路线。

04 我们站在一个需要了解更多哲思的位置

上面说的很多，其实重演了软件工程的历史，对可靠性、可验证性、可扩展性和可维护性的需求是完全相同的。我们见证了乔姆斯基谱系中的权衡在提示与上下文的辩论中重现，又在think tool这样的形式化认知步骤中具象出来。

当前的发展虽然迅速，但仍处于范式形成前的阶段，下一步是为Agent System发展出一套全面的形式化理论，这可能涉及扩展现有的逻辑框架（如LF），或开发新的框架来处理基于LLM的智能体所固有的概率性和动态性，**我们可以预见，未来将出现更多针对不同推理模式的专用认知工具**，例如分析型、创造型的推理工具，最终目标是建立一个系统，其中AI智能体的行为可以被精确规约，并能根据该规约进行形式化验证，就像一个编译器的正确性可以被证明一

样，这对于在金融、医疗等高风险、安全攸关的领域部署自主智能体至关重要。由乔姆斯基开启的语言形式化之旅，如今正进入一个崭新而深刻的阶段，思想本身的形式化。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 3635